WEEKLY MINI PROJECT 03 · TEAM 4

# Who Doesn't Know?

## Work Package · Frontend B

Frontend | React + TS

| PROJECT | TEAM LEADER | OWNER |
|---|---|---|
| **Who Doesn't Know?** | **Taima Watfeh** | **Nawar Hasan** |

| ROLE | AREA |
|---|---|
| **Frontend developer** | **Game screen · Voting screen** |

> *The screen players look at all game, and the moment they point the finger.*

## 1 Your Work at a Glance

| # | Task | Main file(s) |
|---|---|---|
| T1 | Your API file: ask / answer | `api/game.api.ts` |
| T2 | Your API file: vote | `api/voting.api.ts` |
| T3 | Your mock data | `mocks/game.mock.ts`, `mocks/voting.mock.ts` |
| T4 | Role card | `components/game/RoleCard.tsx` |
| T5 | Turn info + Q/A log | `TurnInfo.tsx`, `QaLog.tsx` |
| T6 | Ask form + answer form | `AskForm.tsx`, `AnswerForm.tsx` |
| T7 | Game screen | `screens/GameScreen.tsx` |
| T8 | Voting screen | `screens/VotingScreen.tsx` |
| T9 | Your preview page | `dev/NawarPreview.tsx` |

## 2 Mock Data First

PHASE 1 · DAYS 1–3

### Mocks ON

`VITE_USE_MOCKS=true`. Your api file returns data from **your own** mock file.

PHASE 2 · DAY 4

### Mocks OFF

`VITE_USE_MOCKS=false`. The same api functions call the real backend.

PHASE 3 · DAY 4–5

### Fix together

Run the flows on the real backend; fix any difference in your own files.

## 3 Your Tasks, Step by Step

### T1 Your API file: ask / answer

`src/api/game.api.ts`

1. `askQuestion(session, targetRef, text)` → `POST /games/:roomCode/questions`.
2. `answerQuestion(session, questionId, text)` → `POST /games/:roomCode/questions/:questionId/answer`.
3. Each: `if (USE_MOCKS) return mock.xxx(...)`, else `request(...)`.

```
// Pattern every frontend api file follows
import { request, USE_MOCKS } from './http';
import * as mock from '../mocks/game.mock';
import type { Session } from '../types';

export function askQuestion(s: Session, targetRef: number, text: string): Promise<{ ok: true }> {
  if (USE_MOCKS) return mock.askQuestion(targetRef, text);
  return request('POST', `/games/${s.roomCode}/questions`, { playerId: s.playerId, targetRef,
text });
}
```

✔ Done when: your screens never call `fetch` directly.

### T2 Your API file: vote

`src/api/voting.api.ts`

1. `submitVote(session, suspectRef)` → `POST /games/:roomCode/votes`.

✔ Done when: voting works with mocks on and off.

### T3 Your mock data

`src/mocks/game.mock.ts` · `src/mocks/voting.mock.ts`

1. Fixtures (`PublicGameState` + `PrivateRoleView` pairs): `myTurn`, `askedMe`, `waitingOthers`, `hiddenPlayerTurn`, `afterVoteTie`, `votingNotVoted`, `votingVoted`.
2. Use 5 players: Taima, Ali, Alaa, Nawar, Yazan; a log with 4–6 entries over 2 rounds.
3. Mock functions: `askQuestion` rejects empty text ("Question cannot be empty") and self-target ("You cannot ask yourself"), else `mockDelay({ ok: true })`.
4. `answerQuestion` rejects empty text; `submitVote` rejects self-vote ("You cannot vote for yourself").

✔ Done when: every situation of your two screens has a fixture.

### T4 Role card

`src/components/RoleCard.tsx`

1. `me.role === 'informed'` → big card: "The secret: *{secretTip}*".
2. `'hidden'` → different color: "You don't know the secret. Blend in!".

✔ Done when: both fixtures look clearly different.

## T5 Turn info + Q/A log

`src/components/TurnInfo.tsx` · `src/components/QaLog.tsx`

1. TurnInfo: round number, current player, next player, "{turnsCompletedInRound} done · {turnsRemainingInRound} left".
2. QaLog: grouped by round; each entry "Ali → Alaa: question / answer"; unanswered shows "waiting...".
3. Map refs to names with `state.players`.

✔ Done when: names, never ref numbers, are shown.

## T6 Ask form + answer form

`src/components/AskForm.tsx` · `src/components/AnswerForm.tsx` · `src/components/PlayerPicker.tsx`

1. AskForm: shown only if `state.currentPlayerRef === me.playerRef` and no `pendingQuestion`. PlayerPicker lists everyone **except me**.
2. AnswerForm: shown only if `pendingQuestion.targetRef === me.playerRef`.
3. On submit: call your api function, then `refresh()`. Disable the button while sending; show `error.message` under the form.

✔ Done when: forms appear only for the right player; errors are visible.

## T7 Game screen

`src/pages/GamePage.tsx`

1. Receives `ScreenProps`. Layout: RoleCard, TurnInfo, AskForm / AnswerForm / waiting text, QaLog.
2. Waiting text: "Ali is choosing a question..." or "Waiting for Alaa to answer...".
3. If `state.voting.lastTie` exists, show a banner: "The vote was tied. Keep asking!".
4. Mobile-first layout.

✔ Done when: all game fixtures render correctly on phone width.

## T8 Voting screen

`src/pages/VotingPage.tsx`

1. RoleCard on top; PlayerPicker (everyone except me) + "Vote" button.
2. After voting (`me.hasVoted`): disable and show "You voted. Waiting for others...".
3. "{submittedCount} / {totalPlayers} voted". Never show who voted for whom.

✔ Done when: one vote per player, count visible.

## T9 Your preview page

`src/pages/NawarPreviewPage.tsx`

1. Buttons for each fixture; render GameScreen or VotingScreen with it, a fake `session` and `refresh = () => {}`.

✔ Done when: you can show every state of your screens without the backend or Yazan's routing.

## 4 Shared Files You Import (do not edit)

```
// frontend/src/types.ts  (Day-0 foundation)
export interface Session { roomCode: string; playerId: string; playerRef: number; }
export interface ScreenProps {
  state: PublicGameState;      // from Yazan's polling hook
  me: PrivateRoleView;         // from Yazan's polling hook
  session: Session;            // send session to YOUR OWN api functions
  refresh: () => void;         // call after your action succeeds, to update immediately
}
```

## 5 Rules for You

- Only edit your own files listed above.
- Don't decide whose turn it is: always read `state.currentPlayerRef`.
- Mocks must match `types.ts` field by field.

## 6 Presentation Part

Your part in the English presentation: **live demo of the game screen and the voting screen**. Prepare 1–2 minutes and be ready to explain your own code.